athena.

SPEAKING CARDS · NCRB CRIMES AGAINST WOMEN · 2001–2021

Thirteen cards.
One document story.

A card per slide: the thesis, an opening line, the beats to hit, the number to land, and the transition. The deck follows one state-year fact from the document model to the read model.

13 SLIDES | ~12 MINUTES | NOSQL ONLY | PRINT A4

Numbers to have ready

Documents	736
States / UTs	36
Years	2001–2021
Crime heads	7
Recorded cases	4,867,722
UP 2021 total	2,212
Top state	UP · 529,734
Second	MP · 413,157
Third	WB · 409,242
National 2001	128,537
National 2021	348,092
Growth	+170.8%

Run of show

1. **Cover** 0:45
2. **Why a document store** 1:00
3. **The data grain** 0:50
4. **Document anatomy** 1:00
5. **Collections & schema** 0:55
6. **Write path** 0:55
7. **Indexes** 0:50
8. **Access layer** 1:05
9. **Query console** 1:10
10. **Aggregation** 0:55
11. **Representation** 0:55
12. **Trade-offs & scale** 0:50
13. **Close** 0:40

> **If you have five minutes:** use cards 01, 03, 04, 08, 10 and 13. The through-line is one document holds the fact, one function reads it, one pipeline shapes it, one payload represents it.

CARD 01 · COVER

0:45

Athena.

A deliberately narrow NoSQL walkthrough of one dataset: NCRB crimes against women, 2001–2021.

> SAY
> "Athena is a NoSQL-backed console for NCRB state-level crimes against women, 2001 to 2021. This deck is deliberately narrow: we follow one fact — one state, one year, seven crimes — all the way through the document model, the write path, indexes, the access layer, query and aggregation, and finally representation."

- Point to the document card: "This is the actual shape of one MongoDB document, not a mock-up of the UI."
- The deck is about how NoSQL is used — not a product pitch.
- Four anchors: 736 documents, 36 states and UTs, 7 crime heads, 2001–2021.

LAND

4,867,722 recorded cases, stored as 736 documents.

MOVE

"Let's start with why this data wants to be a document in the first place."

ONE-LINER Athena turns a state-year crime fact into one MongoDB document, then reads it without a join.

CARD 02 · WHY A DOCUMENT STORE

1:00

Why a document store.

The NCRB row is already a complete fact; a document keeps it whole.

> SAY
> "The argument is not that NoSQL is fashionable. It's that the NCRB row is already a complete fact: one state, one year, seven crimes against women."

- A relational schema would split that fact across `records`, `states`, and `crime_types`, then join it back on every read.
- The document model stores the fact once; the read is one fetch.
- Trade-off: shared vocabulary is referenced, not embedded. Labels and colors live in small collections.
- Point to the comparison: "One document versus three tables and a join."

LAND

"The unit of storage matches the unit of meaning."

MOVE

"So what exactly is one document?"

ONE-LINER One state-year fact, one document — no reassembly on read.

CARD 03 · THE DATA GRAIN

0:50

The data grain.

One document = one state or UT × one year × seven crime heads.

> SAY
> "The source is the NCRB state-level table for crimes against women, 2001 to 2021. After normalization, the fact collection has 736 documents."

- 36 states and UTs across 21 years. Early years cover 34 entities, which is why the count is 736, not 756.
- The seven measures: rape, kidnapping and assault, dowry deaths, assault on women, insult to modesty, domestic violence, trafficking.
- `total` is derived per document and stored.
- Caveat: the NCRB reclassified categories after 2014, so large swings around that date may be reporting changes, not incidence.

LAND

736 facts; 4.87 million recorded cases.

MOVE

"Here is what one of those documents actually looks like."

ONE-LINER One state, one year, seven crimes, one stored total.

CARD 04 · DOCUMENT ANATOMY

1:00

Document anatomy.

The document is homogeneous by contract; `state` + `year` is the identity.

> SAY
> "This is the actual MongoDB document for Uttar Pradesh in 2021 — not an illustration."

- `state` and `year` form the composite key; together they are unique.
- The seven crime fields are the measures. Zero is a real observation — "none reported" — not missing data.
- Walk the total: 2,212 cases — 534 rape, 402 kidnapping and assault, 72 dowry deaths, 655 assault on women, 13 insult to modesty, 519 domestic violence, 17 trafficking.
- `_id` is projected out; it is storage, not part of the application contract.
- The shape is identical for all 736 documents. MongoDB does not enforce it — the ETL does.

LAND

One document carries the whole fact; nothing to reassemble on read.

MOVE

"If that's the fact, what else is in the database?"

ONE-LINER The document is the fact, and its shape is guaranteed by the ETL.

CARD 05 · COLLECTIONS & SCHEMA

0:55

Four collections.

One fact collection, two vocabularies, one metadata collection.

> SAY
> "Four collections carry the data layer."

- `crime_records` is the fact collection — 736 documents.
- `states` and `crime_types` are the vocabulary. They exist for inspection and future joins; today the app pulls labels and colors from `CRIME_META` in code.
- `metadata` stores the years and states arrays that the API reads alongside the facts.
- Design rule: embed the measures because they are always read together; reference the vocabulary because duplicating labels into 736 documents makes updates harder.
- Point to the four chips: embed, reference, materialize, validate.

LAND

Embed the fact, reference the vocabulary.

MOVE

"Now, how does a row get in there safely?"

ONE-LINER One fact collection, two vocabularies, one metadata document set.

CARD 06 · WRITE PATH

0:55

Write path.

Normalize once, upsert by the grain; the import is idempotent.

> SAY
> "Two scripts do the work, and the raw CSV is never served to the browser."

- `normalize-csv.mjs` parses the raw CSV, aliases 37 state spellings into 36 canonical entities, validates every numeric field, drops duplicates, and derives `total`.
- `import-mongo.mjs` bulk-writes each record with `replaceOne` and `upsert`, keyed on `{state, year}`, seeds the lookup collections, and creates the indexes.
- Re-running the import does not duplicate facts; the filter is the grain.
- The total is computed once at write time, not on every read.

LAND

One write per fact; idempotent by construction.

MOVE

"That unique key is also the first index."

ONE-LINER Normalize once, upsert by state and year, rerun safely.

CARD 07 · INDEXES

0:50

The grain is the index.

The unique compound index is the integrity rule.

> SAY
> "Three indexes are created by the import script, not at request time."

- Unique compound `{state: 1, year: 1}`: one fact per state-year, duplicate writes rejected, and the upsert stays safe.
- `{year: 1}`: bounded year windows and timeline slices.
- `{state: 1}`: one state across all 21 years, plus the state dossier route.
- Point to the planner panel: "State plus year is an `IXSCAN` then one document fetch; a year range is a bounded scan; a state `$in` is one seek per value."
- Honest note: at 736 documents the app also reads the whole collection once. These are the shapes for direct queries and for scale.

LAND

"Integrity, not just speed."

MOVE

"Indexes only matter if the access layer uses them. Here is the one function every route calls."

ONE-LINER The unique state-year index is what makes the upsert safe.

CARD 08 · ACCESS LAYER

1:05

One function hides the database.

Every route calls `getStore()` and gets the same shape.

> SAY
> "This is the single most important code path in the deck. Every API route calls `getStore()`."

- If `MONGODB_URI` is set, connect with a 4-second server selection timeout. Fail fast; never hang a page request.
- Read `crime_records` with `{_id: 0}`, then read the `metadata` years and states.
- Cache the `Store` in module scope. A warm process does not re-read 736 documents on every request.
- If the URI is missing, the connection fails, or the collection is empty, fall through to bundled JSON. The `source` field records which path answered.
- Store shape: `records`, `years`, `states`, `crimeList`, `source`.

LAND

"One read per process, not per request."

MOVE

"With that layer in place, the query console becomes four native patterns."

ONE-LINER One cached `getStore()`, one shape, with a seed fallback.

CARD 09 · QUERY CONSOLE · INTERACTIVE

1:10

Query documents, not tables.

Four access patterns cover the app; each is a native MongoDB query.

> SAY
> "This slide is interactive. The tabs switch between the four access patterns that cover the app."

- **Point lookup:** `{state: "Uttar Pradesh", year: 2021}` → 1 document, total 2,212, `IXSCAN` on the compound index.
- **Year range:** bounded scan on `{year: 1}` — for example, 2014 to 2021.
- **State filter:** `$in` on a set of states; one seek per value on `{state: 1}`.
- **Top states:** group by state and sum `total`. Uttar Pradesh 529,734; Madhya Pradesh 413,157; West Bengal 409,242.
- No joins, no ORM. The result is already the document shape.

Stage cue: click Year range, then Top states. Leave Point lookup up if you are short on time.

LAND

Four questions, four index paths, zero joins.

MOVE

"The top-states tab is an aggregation. Let's look at the pipeline behind it."

ONE-LINER Point lookup, year range, state filter, top states — all native queries.

CARD 10 · AGGREGATION

0:55

Aggregation is the read model.

The pipeline is match, group, sort, project.

> SAY
> "This is the NoSQL read model at its most useful."

- `$match` clamps the year window before any grouping work.
- `$group` folds all 736 documents into one bucket per year and sums `total`.
- `$sort` puts the 21 output documents in chronological order.
- `$project` returns only `year` and `cases`.
- The result: 21 points. 2001 is 128,537; 2021 is 348,092; that is +170.8%.
- Honest note: at this size the app reads once and folds in process. The pipeline is the server-side path for scale and selective queries.

LAND

One pipeline, 21 points, no client-side number crunching.

MOVE

"The last step is representation: how those documents become one payload."

ONE-LINER Match, group, sort, project — 736 documents into 21 points.

CARD 11 · REPRESENTATION

0:55

Stored as documents. Read as one payload.

JSON all the way down: document → Store → DashboardPayload.

> SAY
> "Representation here is not a UI concern; it's a data-shape concern."

- A BSON document leaves the collection as a plain JavaScript object with the same keys.
- `getStore()` wraps the array in a `Store` with years, states, the crime list, and a source flag.
- The API projects the Store into one `DashboardPayload`: meta, hero, national series, rankings, crime composition, insights.
- No object-relational mapping and no join; the document shape survives from disk to response.
- The payload is UI-shaped, but every array in it is derived from the same 736 documents.

LAND

"One read, one object, one response."

MOVE

"So why is this the right model at this scale — and what changes when it grows?"

ONE-LINER The document shape survives all the way from disk to response.

CARD 12 · TRADE-OFFS & SCALE

0:50

Why this model holds.

Embed, reference, materialize, cache — and scale by extending, not rewriting.

> SAY
> "This is the honest summary of the NoSQL design."

- Embed the measures because they are read together.
- Reference labels and colors because they are shared.
- Materialize `total` and the filter arrays because recomputing them adds no information.
- Cache the Store: at 736 documents, one collection read beats a database round-trip per filter.
- Scale path: shard on state, move aggregation server-side, add district-level documents with the same grain principle.
- What we did not build: a collection per crime head, a join table, or a client-side database.

LAND

Small now, but the grain and the access path scale.

MOVE

"Close on the architecture, not on a claim."

ONE-LINER Embed the fact, reference the vocabulary, cache the read, shard by state later.

CARD 13 · CLOSE

0:40

Close on the architecture.

Model the fact, write it once, index the grain, access through one layer, represent as one payload.

> SAY
> "Close on the architecture, not on a claim."

- Model the state-year fact as one document.
- Write it once with an idempotent upsert.
- Index the grain.
- Access it through one cached Store.
- Represent it as one payload.

LAND

A small document store, a projection-based access layer, and a read model that is JSON all the way down.

MOVE

"Questions?"

ONE-LINER One document, one upsert, one index, one Store, one payload.

Likely questions

"Why not just use Postgres?"

At this size, either works. The document model wins because the fact is already one row-shaped object and the app never needs a join. If the access pattern changed to heavy ad-hoc relational reporting, a relational store would be a fair choice.

"736 documents is tiny. Why talk about indexes and sharding?"

It is tiny, and the deck says so. That is why the app reads once and caches. The indexes and pipeline show the access paths that matter when the collection grows — not a claim that 736 documents need sharding.

"How do you stop bad documents getting in?"

The ETL validates every numeric field and normalizes the shape. MongoDB itself does not enforce the schema here; the contract is enforced at write time. A production version could add MongoDB JSON Schema validation as a second line of defence.

"What about the NCRB data quality caveats?"

These are reported cases, not incidence. The NCRB reclassified categories after 2014, so some swings around that date are reporting changes. Zero means none reported, not missing.

"Where do the dashboard numbers come from?"

The same 736 documents. `getStore()` returns the Store, the API projects it into one DashboardPayload, and every chart is derived from that payload. There is no separate analytics database.

"Why keep `states` and `crime_types` if the code uses `CRIME_META`?"

They are provisioned for inspection and future joins. Today `CRIME_META` is the source of labels and colors; the collections make the vocabulary explicit and give the data layer a place to grow.